Design Report: Custom 4-Bit CPU in Logisim

Integrating a 4-bit ALU, a local register file, a 32 x 4 data RAM and a custom ISA

Digital Logic and Computer Organization Laboratory · Computer Architecture and Digital Design Laboratory

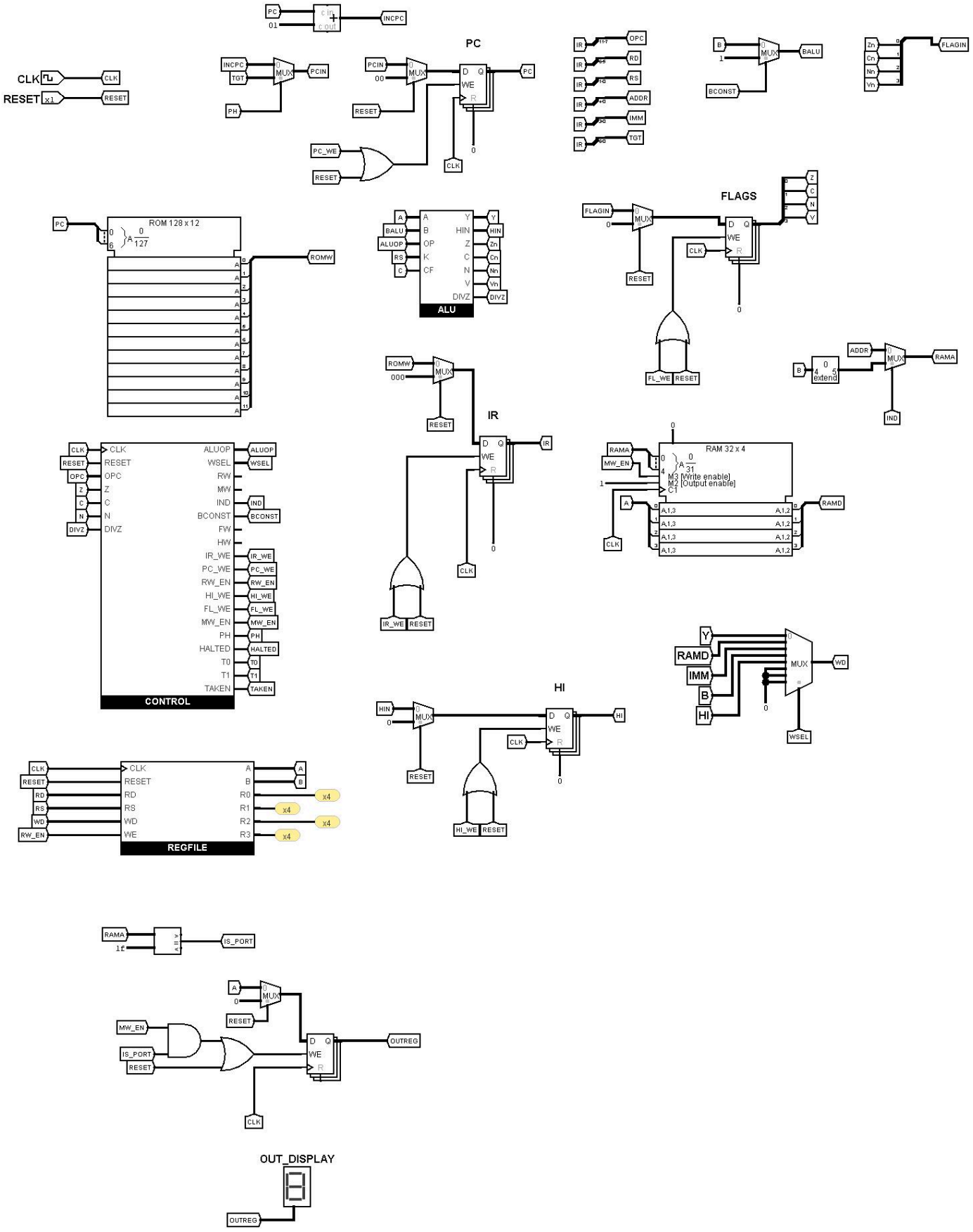
Field	Entry
Name	Prav Prateek, Prakhar Kanoujia, Prakriti Minz
Roll number	25JE0948, 25JE0946, 25JE0947
Group	3
Project	Custom 4-Bit CPU Design
Date of submission	01-11-2026
Circuit file	25JE0946_CPU.circ
Logisim version	v4.1.0

1. Overview and Objectives

This report describes a programmable 4-bit CPU built in Logisim from the modules developed in earlier sessions: a 4-bit ALU, a register file, multiplexers, registers and a RAM. The CPU is a synchronous sequential circuit driven by one master clock. It fetches 12-bit instructions from a program ROM, decodes control signals from a control ROM, reads and writes operands in four local registers and a 32 x 4 data RAM, and executes arithmetic, logic, shift, divide, compare and branch operations.

1.1 Required features and extras

Requirement	How it is met	Implemented in my circuit
Six baseline ALU operations: ADD, SUB, MUL, AND, OR, XOR	Parallel units inside the ALU, selected by ALU_Op	[✓]
Flags Z, C, N, V	FLAGS register updated from the ALU	[✓]
2 to 4 local registers	R0 to R3 (4 bits each)	[✓]
Extended result register	HI holds the upper nibble of MUL, the remainder of DIV, the overflow of SHL and the shifted-out bits of SHR	[✓]
PC, IR, FLAGS	PC (7 bits), IR (12 bits), FLAGS (4 bits)	[✓]
2^n x 4 RAM with LOAD and STORE	32 x 4 RAM (n = 5)	[✓]
Own ISA	31 instructions, 12-bit word	[✓]
Subcircuits ALU, register file, control unit, top level	ALU, REGFILE, CONTROL, CPU_TOP	[✓]
Enhancement: shift unit	SHL, SHR, ROL, ROR	[✓]
Enhancement: hardware division	DIV with remainder and divide-by-zero flag	[✓]
Enhancement: branching and loops	JMP, JZ, JNZ, JC, JN	[✓]
Enhancement: chained arithmetic	ADC, SBB	[✓]
Enhancement: comparison and bit manipulation	CMP, NOT, ROL, ROR	[✓]
Supporting instructions	INC, DEC, MOV, MFHI, LDX, STX (register-indirect), NOP, HALT	[✓]
Optional extras	Output display	[✓]



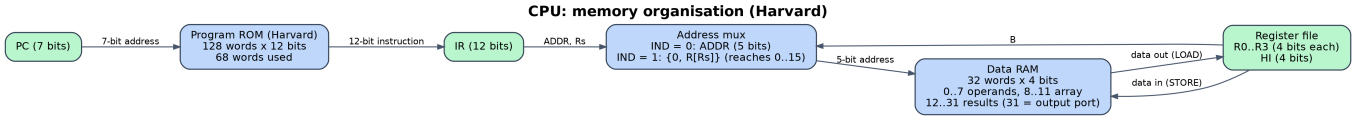
2. Architecture Decisions and Derivations

2.1 Parameters

Parameter	Value	Derivation
Instruction count M	31	15 control, data-move and branch instructions, 15 ALU instructions, 1 HALT
Opcode bits	5	N_opcode = ceil(log2 31) = 5 (32 codes, one spare)
Local registers K	4 (R0 to R3) plus HI	N_reg = ceil(log2 4) = 2 bits per register field
RAM address bits n	5	A 32 x 4 RAM holds 8 operands, a 4-element array and 20 result slots, about 32 nibbles. The address field then fits exactly in the instruction word
Instruction width W	12	W = N_opcode + N_reg + N_addr = 5 + 2 + 5 = 12
Program ROM	128 x 12	A jump target uses 7 bits (12 minus the 5-bit opcode), so PC has 7 bits and the ROM has 2^7 = 128 words. The test program uses 68 words
Flags	Z, C, N, V	4 bits
Data width	4	Registers, ALU, RAM data

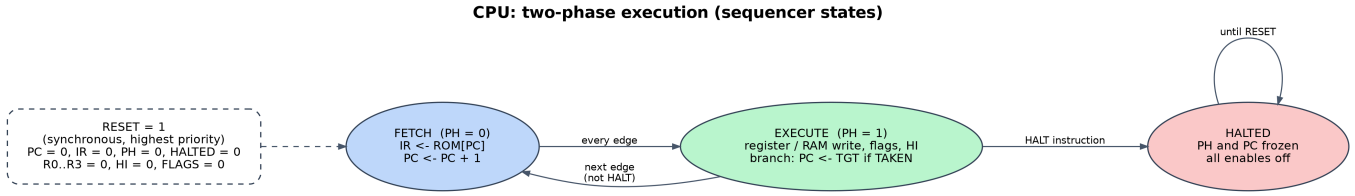
2.2 Memory organisation: Harvard

A separate instruction ROM (12-bit words) and a 32 x 4 data RAM were chosen. Reasons: (1) a 12-bit instruction is fetched in a single cycle, whereas a shared 4-bit memory would need three fetch cycles per instruction; (2) instruction and data accesses never compete for one bus; (3) the ROM holds the program image and the RAM holds the operands and results cleanly.



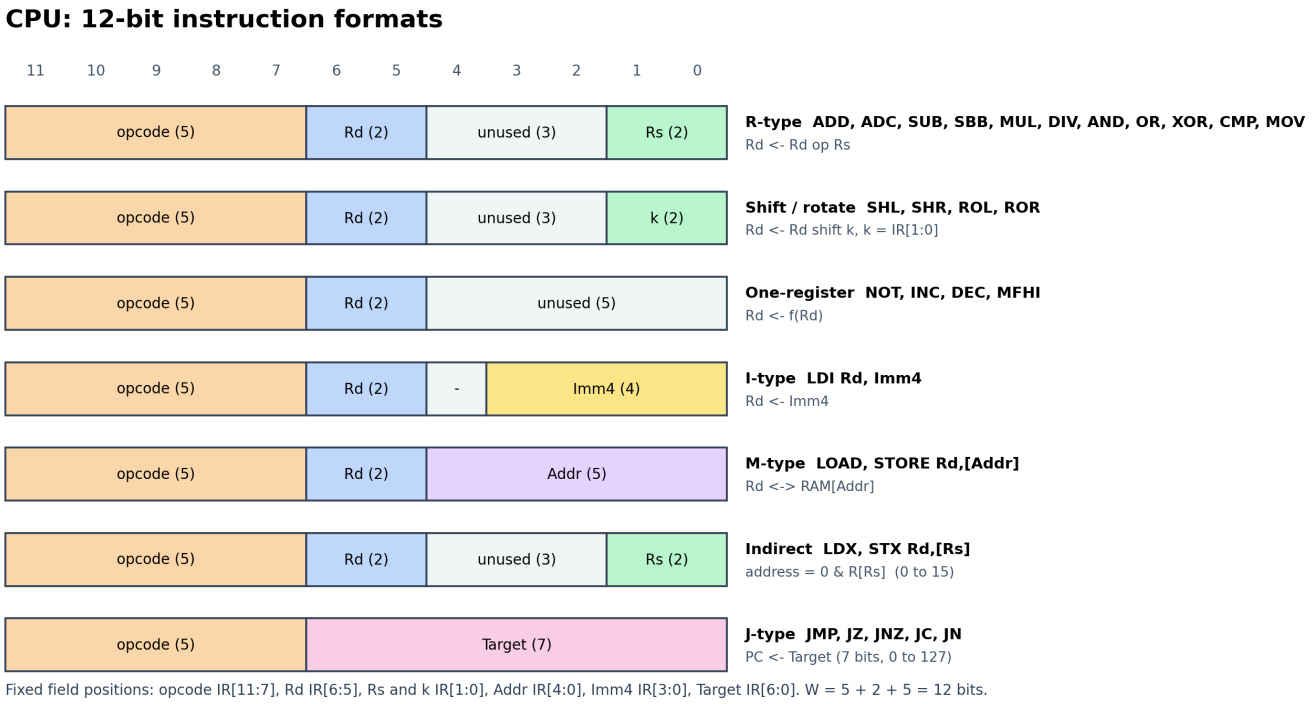
2.3 Execution model: two-phase multi-cycle

A phase flip-flop PH selects FETCH (PH = 0) or EXECUTE (PH = 1). In FETCH, IR <- ROM[PC] and PC <- PC + 1. In EXECUTE the register write, RAM write, flags, HI and branch take place. Every instruction takes exactly two clock edges (CPI = 2). Fetch and execute are separated so a register or the RAM is never read and written on the same edge, and the probes show each phase.



3. Instruction Set Architecture

3.1 Instruction formats



Fixed field positions: opcode IR[11:7], Rd IR[6:5], Rs and k IR[1:0], Addr IR[4:0], Imm4 IR[3:0], Target IR[6:0]. W = 5 + 2 + 5 = 12 bits.

Fields are taken from fixed IR positions: opcode IR[11:7], Rd IR[6:5], Rs and k IR[1:0], Addr IR[4:0], Imm4 IR[3:0], Target IR[6:0]. The decoder decides how each field is used.

3.2 Opcode and RTL table

Opcodes are hexadecimal (5 bits). ALU-group opcodes are 10 to 1E with ALU_Op = opcode[3:0].

Opcode	Mnemonic	Syntax	RTL	Flags
00	NOP	NOP	none	none
01	LOAD	LOAD Rd,[Addr]	Rd <- RAM[Addr]	none
02	STORE	STORE Rd,[Addr]	RAM[Addr] <- Rd	none
03	LDI	LDI Rd,Imm4	Rd <- Imm4	none
04	MOV	MOV Rd,Rs	Rd <- Rs	none
05	MFHI	MFHI Rd	Rd <- HI	none
06	LDX	LDX Rd,[Rs]	Rd <- RAM[0 & Rs]	none
07	STX	STX Rd,[Rs]	RAM[0 & Rs] <- Rd	none
08	JMP	JMP Target	PC <- Target	none
09	JZ	JZ Target	if Z then PC <- Target	none
0A	JNZ	JNZ Target	if NOT Z then PC <- Target	none
0B	JC	JC Target	if C then PC <- Target	none
0C	JN	JN Target	if N then PC <- Target	none
0D	INC	INC Rd	Rd <- Rd + 1	Z C N V
0E	DEC	DEC Rd	Rd <- Rd - 1	Z C N V
0F	reserved		(acts as NOP, used for CALL and RET if implemented)	
10	ADD	ADD Rd,Rs	Rd <- Rd + Rs	Z C N V
11	ADC	ADC Rd,Rs	Rd <- Rd + Rs + C	Z C N V
12	SUB	SUB Rd,Rs	Rd <- Rd - Rs	Z C(borrow) N V

Opcode	Mnemonic	Syntax	RTL	Flags
13	SBB	SBB Rd,Rs	Rd <- Rd - Rs - C(borrow)	Z C N V
14	MUL	MUL Rd,Rs	HI:Rd <- Rd x Rs	Z C V N
15	DIV	DIV Rd,Rs	Rd <- Rd / Rs, HI <- Rd mod Rs	Z N V(div0)
16	AND	AND Rd,Rs	Rd <- Rd AND Rs	Z N, C=V=0
17	OR	OR Rd,Rs	Rd <- Rd OR Rs	Z N, C=V=0
18	XOR	XOR Rd,Rs	Rd <- Rd XOR Rs	Z N, C=V=0
19	NOT	NOT Rd	Rd <- NOT Rd	Z N, C=V=0
1A	SHL	SHL Rd,k	HI:Rd <- {0000,Rd} << k	Z N C V
1B	SHR	SHR Rd,k	Rd <- Rd >> k (logical), HI <- shifted-out bits	Z N C
1C	ROL	ROL Rd,k	Rd <- rotate left Rd by k	Z N
1D	ROR	ROR Rd,k	Rd <- rotate right Rd by k	Z N
1E	CMP	CMP Rd,Rs	flags <- Rd - Rs, no write-back	Z C N V
1F	HALT	HALT	freeze PC and phase	none

3.3 Flag conventions

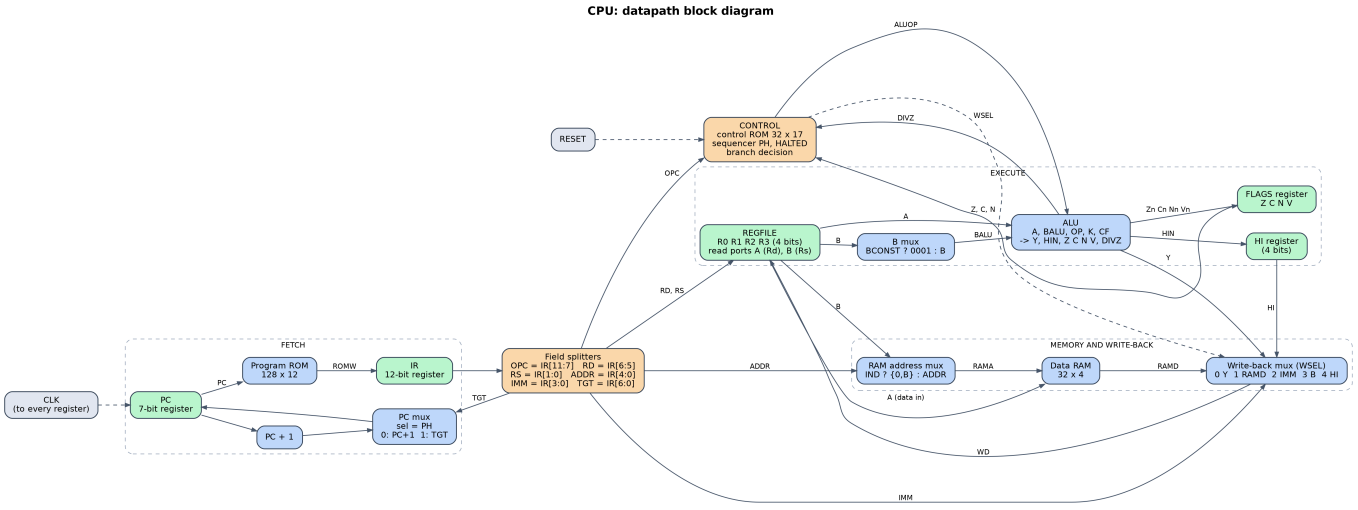
For ADD, ADC and INC, C is the carry out. For SUB, SBB, CMP and DEC, C is the **borrow** (C=1 means Rd < Rs unsigned), so JC after CMP means less-than. N is always Result[3]. Z means the 4-bit result is zero (for MUL the whole 8-bit product). V is signed overflow for the add and subtract family, (HI not zero) for MUL and SHL, and divide-by-zero for DIV. MOV, LDI, LOAD, STORE, MFHI and branches leave the flags unchanged.

---

4. Datapath

4.1 Block diagram

```
FETCH    :  PC --> program ROM --> ROMW --> IR
DECODE   :  IR --> field splitters --> OPC, RD, RS, ADDR, IMM, TGT
CONTROL  :  OPC, flags Z C N, DIVZ --> CONTROL --> ALUOP, WSEL, IND, BCONST and all enables
EXECUTE  :  REGFILE(RD, RS) --> A, B --> ALU(A, BALU, ALUOP, K=RS, CF=C) --> Y, HIN, flags
WRITE    :  WD mux (Y, RAMD, IMM, B, HI) --> REGFILE ;  HIN --> HI ;  flags --> FLAGS ;  A --> RAM data in
```



4.2 Components

Block	Implementation
PC	7-bit Register, input mux: PC + 1 (fetch) or IR[6:0] (taken branch)
Program ROM	Logisim ROM, 7 address bits, 12 data bits, address = PC
IR	12-bit Register, loaded in FETCH
Register file	Four 4-bit Registers R0 to R3, 2-to-4 decoder gated by RegWrite, two 4:1 read multiplexers (port A selects Rd = IR[6:5], port B selects Rs = IR[1:0])
Write-back mux	8:1 mux (WSEL): 0 Y, 1 RAM data, 2 Imm4, 3 B, 4 HI
B-operand mux	B_alu = BCONST ? 0001 : B (INC and DEC)
ALU	Subcircuit, inputs A, B, OP, K, CF, outputs Y, HIN, Z, C, N, V, DIVZ
HI register	4-bit Register, enable HI_WE
FLAGS register	4-bit Register {V,N,C,Z}, enable FL_WE
Data RAM	32 x 4, address = IND ? {0,B} : IR[4:0], data in = A, write enable = MW_EN
Control ROM	32 x 17 bits addressed by the opcode
Sequencer	PH and HALTED flip-flops

Every register uses a synchronous reset: its enable is OR(write signal, RESET) and its D input is selected between the data and zero by RESET. Reset sets PC=0, IR=0 (NOP), PH=0, HALTED=0, R0 to R3=0, HI=0, FLAGS=0. The RAM and ROM images are not changed by reset.

---

5. Arithmetic Logic Unit

The ALU builds every functional unit in parallel and selects the result with a 16:1, 4-bit multiplexer on ALU_Op. Inputs: A (4), B (4), OP (4), K (2), CF (1). Outputs: Y (4), HIN (4), Z, C, N, V, DIVZ.

ALU_Op	Operation	Y source
0000	ADD	adder sum S
0001	ADC	S
0010	SUB	S
0011	SBB	S
0100	MUL	product low nibble
0101	DIV	quotient
0110	AND	A AND B
0111	OR	A OR B
1000	XOR	A XOR B
1001	NOT	NOT A
1010	SHL	8-bit shifter low nibble
1011	SHR	right shifter output
1100	ROL	rotate-left output
1101	ROR	rotate-right output
1110	CMP	S (write-back disabled by the control word)

5.1 Adder and subtractor

Subtraction uses two's complement: A - B = A + NOT B + 1 on the same adder.

```
sub_mode = (ALU_Op == 0010) OR (ALU_Op == 0011) OR (ALU_Op == 1110)       ; SUB, SBB, CMP (DEC uses 0010)
B'       = B_alu XOR {4{sub_mode}}
cin      = (op==ADC AND C) OR (op==SBB AND NOT C) OR (op==SUB) OR (op==CMP)
{Cout,S} = A + B' + cin
C_new    = Cout XOR sub_mode                    ; carry for add, borrow = NOT Cout for subtract
V        = NOT(A3 XOR B'3) AND (A3 XOR S3)      ; operands of the same sign, result sign differs
```

For SBB, A - B - borrow = A + NOT B + (1 - borrow), so cin = NOT C. For ADC, cin = C.

Worked checks: 5 - 3: 0101 + 1100 + 1 = 1_0010, S=2, Cout=1, so borrow 0. 3 - 5: 0011 + 1010 + 1 = 0_1110, S=E, Cout=0, so borrow 1 and N=1. 7 + 3: 0111 + 0011 = 1010, signed overflow, V=1 and N=1.

5.2 Multiplier

A Logisim Multiplier set to 4 bits has a product output (low nibble) and a carry-out output carrying the upper nibble. So P[7:0] = {cout, product}: Y = product, HIN = cout. Flags: Z = (P[7:0] == 0), C = V = (HIN != 0), N = Y3. Example: 7 x 3 = 21 = 0x15, so Y = 5 and HI = 1.

5.3 Divider and divide-by-zero

```
divz  = NOR(B3,B2,B1,B0)
B_div = divz ? 0001 : B                  ; keeps the divider outputs free of unknown values
Y     = quotient(A, B_div)      HIN = remainder(A, B_div)
flags: Z = (Y==0), N = Y3, C = 0, V = divz
write gating: if (DIV AND divz) then the register write and HI write are suppressed
```

So a divide by zero leaves Rd and HI unchanged and raises V, which a program can test. Example: 12 / 5 gives Y = 2 and HI = 2.

5.4 Logic operations

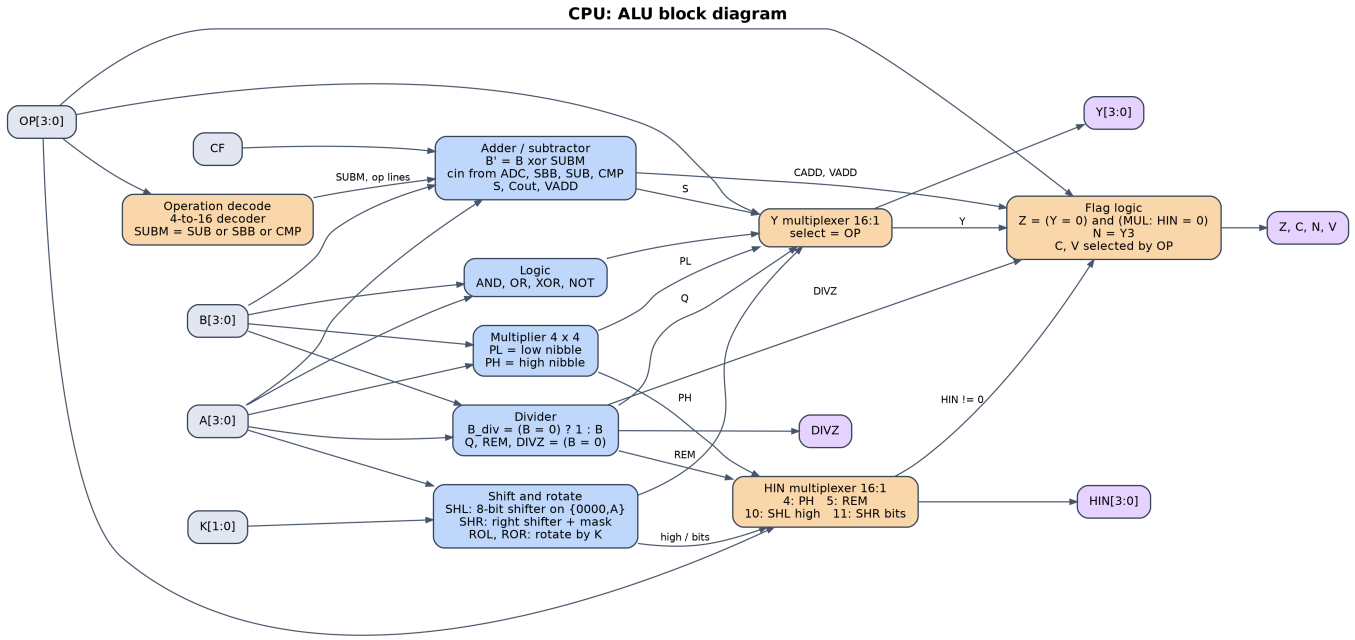
Y = A AND B, A OR B, A XOR B or NOT A. Flags: Z = (Y==0), N = Y3, C = V = 0.

5.5 Shift and rotate unit

The shift amount k = IR[1:0] (0 to 3). SHL uses an 8-bit logical-left shifter on {0000, A} with distance {0, k}: the low nibble is Y and the high nibble is the overflow, so HI:Y = A x 2^k exactly. SHR uses a 4-bit logical-right shifter: Y = floor(A / 2^k), and HI = A AND mask_k holds the shifted-out bits (mask_k = 0000, 0001, 0011, 0111 for k = 0 to 3). ROL and ROR use 4-bit rotate shifters. C = (HI != 0) for SHL and SHR.

5.6 Flag selection

Op	Z	N	C	V
ADD, ADC, INC	Y==0	Y3	Cout	overflow formula
SUB, SBB, CMP, DEC	Y==0	Y3	NOT Cout (borrow)	overflow formula
MUL	P[7:0]==0	Y3	HI!=0	HI!=0
DIV	Y==0	Y3	0	divz
AND, OR, XOR, NOT	Y==0	Y3	0	0
SHL, SHR	Y==0	Y3	HI!=0	HI!=0 for SHL, 0 for SHR
ROL, ROR	Y==0	Y3	0	0



---

6. Control Unit

6.1 Sequencer

Two flip-flops, updated on the master rising edge with synchronous RESET of highest priority:

```
PH_next     = 0                       if RESET
            = PH                      if HALTED           ; frozen
            = NOT PH                  otherwise           ; FETCH <-> EXECUTE
HALTED_next = 0 if RESET ; 1 if (PH AND HALT bit) ; else HALTED
T0 = NOT PH AND NOT HALTED            ; fetch phase active
T1 = PH AND NOT HALTED                ; execute phase active
```

6.2 Control word and control truth table

The control ROM has 32 entries (addressed by IR[11:7]) of 17 bits. Bit order from MSB to LSB: HALT(1), BR(3), BCONST(1), IND(1), HW(1), FW(1), MW(1), WSEL(3), RW(1), ALUOP(4).

BR codes: 000 none, 001 JMP, 010 JZ, 011 JNZ, 100 JC, 101 JN. WSEL: 000 ALU result Y, 001 RAM data, 010 Imm4, 011 B (register Rs), 100 HI.

Opcode	Instr	RW	WSEL	MW	IND	BCONST	FW	HW	BR	ALUOP
00	NOP	0	000	0	0	0	0	0	000	0000
01	LOAD	1	001	0	0	0	0	0	000	0000
02	STORE	0	000	1	0	0	0	0	000	0000
03	LDI	1	010	0	0	0	0	0	000	0000
04	MOV	1	011	0	0	0	0	0	000	0000
05	MFHI	1	100	0	0	0	0	0	000	0000
06	LDX	1	001	0	1	0	0	0	000	0000
07	STX	0	000	1	1	0	0	0	000	0000
08	JMP	0	000	0	0	0	0	0	001	0000
09	JZ	0	000	0	0	0	0	0	010	0000
0A	JNZ	0	000	0	0	0	0	0	011	0000

Opcode	Instr	RW	WSEL	MW	IND	BCONST	FW	HW	BR	ALUOP
0B	JC	0	000	0	0	0	0	0	100	0000
0C	JN	0	000	0	0	0	0	0	101	0000
0D	INC	1	000	0	0	1	1	0	000	0000
0E	DEC	1	000	0	0	1	1	0	000	0010
10	ADD	1	000	0	0	0	1	0	000	0000
11	ADC	1	000	0	0	0	1	0	000	0001
12	SUB	1	000	0	0	0	1	0	000	0010
13	SBB	1	000	0	0	0	1	0	000	0011
14	MUL	1	000	0	0	0	1	1	000	0100
15	DIV	1	000	0	0	0	1	1	000	0101
16	AND	1	000	0	0	0	1	0	000	0110
17	OR	1	000	0	0	0	1	0	000	0111
18	XOR	1	000	0	0	0	1	0	000	1000
19	NOT	1	000	0	0	0	1	0	000	1001
1A	SHL	1	000	0	0	0	1	1	000	1010
1B	SHR	1	000	0	0	0	1	1	000	1011
1C	ROL	1	000	0	0	0	1	0	000	1100
1D	ROR	1	000	0	0	0	1	0	000	1101
1E	CMP	0	000	0	0	0	1	0	000	1110
1F	HALT	0	000	0	0	0	0	0	000	0000, HALT bit = 1
0F	reserved	all zero (acts as NOP)								

The control ROM image in hexadecimal (opcodes 00 to 1F in order) is given in Appendix B.

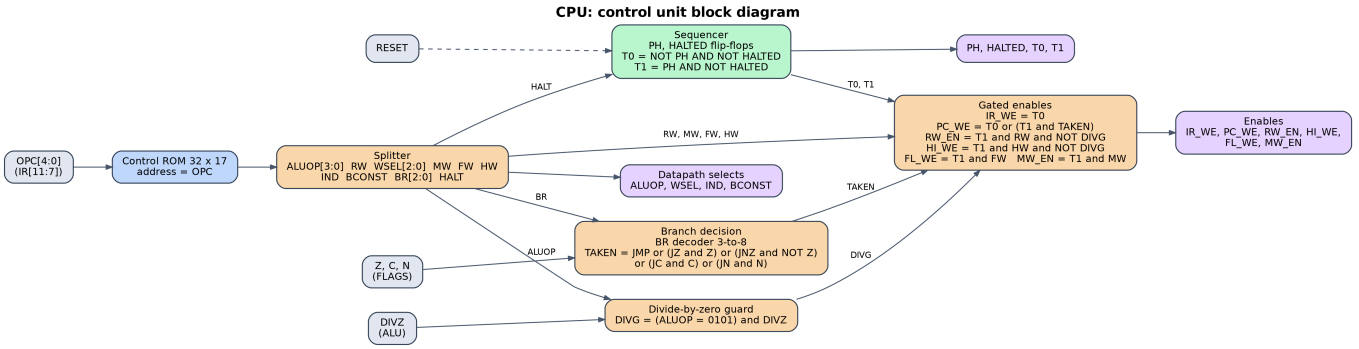
6.3 Enable, branch and multiplexer equations

```
divz_gate = (ALUOP == 0101) AND DIVZ                ; only DIV has ALUOP 0101
IR_WE     = T0
PC_WE     = T0 OR (T1 AND TAKEN)
PC_next   = T0 ? PC + 1 : IR[6:0]                   ; RESET forces 0 (synchronous)
RW_EN     = T1 AND RW AND NOT divz_gate
HI_WE     = T1 AND HW AND NOT divz_gate
FL_WE     = T1 AND FW
MW_EN     = T1 AND MW
TAKEN     = (BR==001) OR (BR==010 AND Z) OR (BR==011 AND NOT Z) OR (BR==100 AND C) OR (BR==101 AND N)
RAM_addr  = IND ? {0, B} : IR[4:0]
Register write: R[Rd] <- write-back mux, decoder(Rd) AND RW_EN
```

Because the PC was already incremented in FETCH, a taken branch overwrites it with the target in EXECUTE. RESET forces PC=0, IR=0, PH=0, HALTED=0 and zeroes the registers, HI and FLAGS.

6.4 Timing of one instruction (ADD R2,R1 at address 03)

Edge	Phase	Action
1	FETCH	IR <- 841 (ROM[03]), PC <- 04, PH <- 1
2	EXECUTE	A = R2, B = R1, Y = A + B, R2 <- Y, FLAGS updated, PH <- 0



7. Enhancements and Their Benefits

Enhancement	Instructions	Hardware	Benefit
Power-of-2 shift unit	SHL, SHR (distance 0 to 3)	8-bit logical-left shifter on {0000, A}; 4-bit logical-right shifter; mask multiplexer for the shifted-out bits	Multiplies or divides by 2, 4 or 8 without the multiplier or divider, with the full 8-bit result in HI:Rd for SHL and the remainder in HI for SHR
Rotate and bit manipulation	ROL, ROR, NOT	4-bit rotate shifters, inverter bank	Bit rotation and complement in one execute cycle
Hardware integer division	DIV	4-bit divider, zero guard	Quotient in Rd and remainder in HI; divide-by-zero sets V and leaves state unchanged so software can test it safely
Conditional branching and loops	JMP, JZ, JNZ, JC, JN	Condition decoder TAKEN from BR and FLAGS into the PC multiplexer	Loops, such as the array-sum loop of the test program
Chained multi-nibble arithmetic	ADC, SBB	cin multiplexing from the C flag	8-bit add and subtract across register pairs, for example (R1:R0) + (R3:R2)
Comparison	CMP	the same adder, write-back disabled	Sets the flags for branches without overwriting operands
Addressing and loop helpers	LDX, STX, INC, DEC, MFHI	address multiplexer {0,B}, B forced to 1, WSEL for HI	Walk arrays with a pointer register, one-instruction loop counters, access to HI
Control	NOP, HALT	HALTED flip-flop gating PH and all enables	Freezes the machine so the final state can be inspected

Performance argument. A shift is a wiring-level operation: a few multiplexer stages, against an array multiplier or a divider with many adder cells. It has a much shorter propagation delay and far fewer gates. In this design MUL and DIV are combinational and take one execute cycle, so SHL and SHR do not save cycles here; the benefit is delay and hardware cost, and the side results in HI. If the multiplier were iterative (shift-and-add, five execute cycles), SHL by k would take one cycle against five. [If you implemented the iterative multiplier extra, give your measured cycle counts here.]

7.1 Optional extras

Extra	Built
Memory-mapped output display (address 31 drives a seven-segment display)	[✓]

8. Self-Contained Verification Program

8.1 Data RAM image (32 x 4, addresses decimal, data hex)

Address	0	1	2	3	4	5	6	7	8	9	10	11	12 to 31
Value	7	3	5	C	B	A	9	3	1	2	3	4	0 (results)

Meaning: X=7, Y=3, Z=5, W=C (12), the 8-bit operands 0xAB (RAM[5]:RAM[4]) and 0x39 (RAM[7]:RAM[6]), and a 4-element array 1, 2, 3, 4 at addresses 8 to 11. Results go to addresses 12 to 31.

8.2 Program ROM image (128 x 12, 68 words used)

Columns: address (hex), machine word (hex), assembly. Word = opcode x 0x80 + Rd x 0x20 + field.

```
00 080 LOAD  R0,[0]        ; X
01 0A1 LOAD  R1,[1]        ; Y
02 240 MOV   R2,R0
03 841 ADD   R2,R1         ; 7+3 = A
04 14C STORE R2,[12]
05 240 MOV   R2,R0
06 941 SUB   R2,R1         ; 7-3 = 4
07 14D STORE R2,[13]
08 240 MOV   R2,R0
09 A41 MUL   R2,R1         ; 21 = 0x15: Rd=5, HI=1
0A 14E STORE R2,[14]
0B 2E0 MFHI  R3
0C 16F STORE R3,[15]
0D 240 MOV   R2,R0
0E B41 AND   R2,R1         ; 3
0F 150 STORE R2,[16]
10 240 MOV   R2,R0
11 BC1 OR    R2,R1         ; 7
12 151 STORE R2,[17]
13 240 MOV   R2,R0
14 C41 XOR   R2,R1         ; 4
15 152 STORE R2,[18]
16 240 MOV   R2,R0         ; (X+Y)*Z, register reuse
17 841 ADD   R2,R1         ; A
18 0E2 LOAD  R3,[2]        ; Z=5
19 A43 MUL   R2,R3         ; 50 = 0x32: Rd=2, HI=3
1A 153 STORE R2,[19]
1B 2E0 MFHI  R3
1C 174 STORE R3,[20]
1D 240 MOV   R2,R0         ; (X xor Y) - Z
1E C41 XOR   R2,R1         ; 4
1F 0E2 LOAD  R3,[2]        ; 5
20 943 SUB   R2,R3         ; 4-5 = F, borrow=1
21 155 STORE R2,[21]
22 083 LOAD  R0,[3]        ; W=12
23 0A2 LOAD  R1,[2]        ; 5
24 A81 DIV   R0,R1         ; Q=2, HI=R=2
25 116 STORE R0,[22]
26 2E0 MFHI  R3
27 177 STORE R3,[23]
28 083 LOAD  R0,[3]        ; 0xC
29 D02 SHL   R0,2          ; 0xC<<2 = 0x30: Rd=0, HI=3
2A 118 STORE R0,[24]
2B 2A0 MFHI  R1
2C 139 STORE R1,[25]
2D 083 LOAD  R0,[3]
2E D82 SHR   R0,2          ; 0xC>>2 = 3, HI = 0
2F 11A STORE R0,[26]
30 084 LOAD  R0,[4]        ; 8-bit add: 0xAB + 0x39
31 0A5 LOAD  R1,[5]
32 0C6 LOAD  R2,[6]
33 0E7 LOAD  R3,[7]
34 802 ADD   R0,R2         ; B+9 = 0x14: R0=4, C=1
35 8A3 ADC   R1,R3         ; A+3+1 = E
36 11B STORE R0,[27]
37 13C STORE R1,[28]
38 180 LDI   R0,0          ; array-sum loop
39 1A8 LDI   R1,8          ; pointer
3A 1C4 LDI   R2,4          ; count
```

```
3B 361 LDX   R3,[R1]       ; loop
3C 803 ADD   R0,R3
3D 6A0 INC   R1
3E 740 DEC   R2
3F 53B JNZ   3B
40 11D STORE R0,[29]
41 18F LDI   R0,F
42 11F STORE R0,[31]       ; done marker
43 F80 HALT
```

8.3 Expected and observed final RAM (hex)

Address	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**	**20**	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**31**
Expected	A	4	5	1	3	7	4	2	3	F	2	2	0	3	3	4	E	A	F
Observed																			
Meaning	add	sub	mul lo	mul hi	and	or	xor	(X+Y)Z lo	hi	(X xor Y)-Z	div Q	div R	shl lo	shl hi	shr	adc lo	adc hi	array sum	done

8.4 What the program demonstrates

Item	**Where**
Loads of operands from RAM into registers	addresses 00, 01, 18, 1F, 22, 23, 28, 2D, 30 to 33
All six baseline operations ADD, SUB, MUL, AND, OR, XOR	addresses 03, 06, 09, 0E, 11, 14
Local register reuse across multi-step calculations	(X+Y)*Z at 16 to 1C and (X xor Y)-Z at 1D to 21
HI register (MUL, DIV, SHL)	MFHI at 0B, 1B, 26, 2B
Custom instructions DIV, SHL, SHR	24, 29, 2E
Chained 8-bit arithmetic ADD then ADC	34, 35
Loop with register-indirect addressing and branch	3B to 3F
Results written back to RAM	all STORE instructions
HALT	43

The program executes 79 instructions (56 straight-line, 3 setup, 16 loop-body, 4 tail), which is 158 clock edges at two edges per instruction.

9. Verification Records

Record stimuli, relevant edges, expected values, observed values and pass or fail for every row. Reset is applied for at least three rising edges before each run.

9.1 Clock settings

Setting	**Value**
Logisim version	v4.1.0
Clock High and Low Duration	1 tick each
Single-stepping	Ctrl+T, two ticks per rising edge

9.2 ALU tests (set OP, A, B, K, CF and read Y, HIN and the flags)

Test	OP	A	B	Expected	Observed	Pass / Fail
ADD carry	0	F	1	Y=0, Z=1, C=1, N=0, V=0	Y=0, Z=1, C=1, N=0, V=0	Pass
ADD signed overflow	0	7	1	Y=8, Z=0, C=0, N=1, V=1	Y=8, Z=0, C=0, N=1, V=1	Pass
ADD both negative	0	8	8	Y=0, Z=1, C=1, V=1	Y=0, Z=1, C=1, V=1	Pass
ADC with carry in	1	5	2 (CF=1)	Y=8	Y=8	Pass
SUB equal	2	5	5	Y=0, Z=1, C(borrow)=0	Y=0, Z=1, C(borrow)=0	Pass
SUB borrow	2	3	5	Y=E, N=1, C=1, V=0	Y=E, N=1, C=1, V=0	Pass
SUB overflow	2	8	1	Y=7, C=0, V=1	Y=7, C=0, V=1	Pass
SBB, borrow in	3	5	2 (CF=1)	Y=2	Y=2	Pass
SBB, no borrow in	3	5	2 (CF=0)	Y=3	Y=3	Pass
MUL max	4	F	F	Y=1, HIN=E, C=1, V=1, Z=0	Y=1, HIN=E, C=1, V=1, Z=0	Pass
MUL zero	4	0	5	Y=0, HIN=0, Z=1	Y=0, HIN=0, Z=1	Pass
DIV	5	C	5	Y=2, HIN=2, V=0	Y=2, HIN=2, V=0	Pass
DIV by zero	5	7	0	DIVZ=1, V=1	DIVZ=1, V=1	Pass
AND	6	A	6	Y=2, C=V=0	Y=2, C=V=0	Pass
OR	7	A	6	Y=E	Y=E	Pass
XOR	8	A	6	Y=C	Y=C	Pass
NOT	9	5	x	Y=A, N=1	Y=A, N=1	Pass
SHL by 1	10	9 (K=1)	x	Y=2, HIN=1, C=1, V=1	Y=2, HIN=1, C=1, V=1	Pass
SHL by 3	10	9 (K=3)	x	Y=8, HIN=4	Y=8, HIN=4	Pass
SHR by 2	11	B (K=2)	x	Y=2, HIN=3, C=1, V=0	Y=2, HIN=3, C=1, V=0	Pass
ROL	12	9 (K=1)	x	Y=3	Y=3	Pass
ROR	13	3 (K=1)	x	Y=9	Y=9	Pass
CMP	14	3	5	as SUB 3-5	as SUB 3-5	Pass

9.3 Register file and control unit tests

Test	Stimulus	Expected	Observed	Pass / Fail
Write and read	WE=1, RD=2, WD=7, edge; RD=1, WD=3, edge; WE=0, WD=F, edge	R2=7, R1=3, no change on the last edge	R2=7, R1=3, no change on the last edge	Pass
Read ports	RD=2, RS=1	A=7, B=3	A=7, B=3	Pass
Synchronous reset	RESET=1 with WE=1 and another RD, edge	R0 to R3 all 0	R0 to R3 all 0	Pass
Sequencer	OPC=10 (ADD), first edges	T0 then T1, IR_WE and PC_WE in FETCH, RW_EN and FL_WE in EXECUTE	T0 then T1, IR_WE and PC_WE in FETCH, RW_EN and FL_WE in EXECUTE	Pass
Branch decision	OPC=0A, PH=1, Z=0 and Z=1	TAKEN=1 then 0	TAKEN=1 then 0	Pass
Divide-by-zero gate	OPC=15, PH=1, DIVZ=1	RW_EN=0, HI_WE=0	RW_EN=0, HI_WE=0	Pass

Test	Stimulus	Expected	Observed	Pass / Fail
HALT	OPC=1F, PH=1	HALTED=1 after the edge, PH and enables frozen	HALTED=1 after the edge, PH and enables frozen	Pass
Reset	RESET=1, edge	PH=0, HALTED=0	PH=0, HALTED=0	Pass

9.4 Instruction and bring-up programs

Program	Words from address 0	Expected	Observed	Pass / Fail
P1 add and store (LDI R0,5; LDI R1,3; ADD R0,R1; STORE R0,[12]; HALT)	185 1A3 801 10C F80	RAM[12]=8, R0=8, HALTED=1 after 10 edges	RAM[12]=8, R0=8, HALTED=1 after 10 edges	Pass
P2 loop with branch (LDI R2,3; DEC R2; JNZ 1; STORE R2,[13]; HALT)	1C3 740 501 14D F80	RAM[13]=0, Z=1, HALTED=1 after 18 edges	RAM[13]=0, Z=1, HALTED=1 after 18 edges	Pass
P3 carry (LDI R0,F; LDI R1,1; ADD R0,R1; HALT)	18F 1A1 801 F80	R0=0, Z=1, C=1, N=0, V=0	R0=0, Z=1, C=1, N=0, V=0	Pass
ADD signed overflow	187 1A1 801 F80	R0=8, N=1, V=1	R0=8, N=1, V=1	Pass
SUB borrow (LDI R0,3; LDI R1,5; SUB R0,R1; HALT)	183 1A5 901 F80	R0=E, C=1, N=1	R0=E, C=1, N=1	Pass
MUL max	18F 1AF A01 F80	R0=1, HI=E, C=1, V=1	R0=1, HI=E, C=1, V=1	Pass
DIV by zero	187 1A0 A81 F80	R0=7 unchanged, V=1	R0=7 unchanged, V=1	Pass
SHL	189 D01 F80	R0=2, HI=1	R0=2, HI=1	Pass

9.5 Golden trace of the full program

After RESET (PC=0, PH=0), single-step and record:

Edge	Phase	Expected & Observed after the edge
1	FETCH	IR=080 (LOAD R0,[0]), PC=01, PH=1
2	EXECUTE	R0=7, PH=0, flags 0000
3	FETCH	IR=0A1, PC=02
4	EXECUTE	R1=3
5	FETCH	IR=240 (MOV R2,R0), PC=03
6	EXECUTE	R2=7, flags unchanged
7	FETCH	IR=841 (ADD R2,R1), PC=04
8	EXECUTE	R2=A, Z=0 C=0 N=1 V=1
9	FETCH	IR=14C (STORE R2,[12]), PC=05
10	EXECUTE	RAM[12]=A
13 and 14	SUB R2,R1	R2=4, Z=0 C=0 N=0 V=0
19 and 20	MUL R2,R1	R2=5, HI=1, C=1, V=1
23 and 24	MFHI R3	R3=1
157 and 158	HALT at address 43	HALTED=1, PC=44, everything frozen

The final RAM contents are compared with the expected table in Section 8.3.

9.6 Optional extras tests (fill if built)

Extra	Test	Expected	Observed	Pass / Fail
Output display	STORE R0, [31]	display shows the value (F at the end of the full program)	display shows the value (F at the end of the full program)	Pass

10. Timing and Performance

Item	Value
Cycles per instruction	2 (one FETCH edge, one EXECUTE edge)
Instructions executed by the verification program	79
Clock edges to HALT	158
Run time at 64 Hz (if used)	about 2.5 s

11. Conclusion

[Write a short summary: what was built, the main results observed, and any limitation or lesson.]

Appendix A: Subcircuit Pin Lists

Subcircuit	Inputs	Outputs
ALU	A (4), B (4), OP (4), K (2), CF	Y (4), HIN (4), Z, C, N, V, DIVZ
REGFILE	CLK, RESET, RD (2), RS (2), WD (4), WE	A (4), B (4), R0, R1, R2, R3 (4 each)
CONTROL	CLK, RESET, OPC (5), Z, C, N, DIVZ	ALUOP (4), WSEL (3), RW, MW, IND, BCONST, FW, HW, IR_WE, PC_WE, RW_EN, HI_WE, FL_WE, MW_EN, PH, HALTED, T0, T1, TAKEN
CPU_TOP	CLK (Clock component), RESET	probes only

Appendix B: Memory Images

Control ROM (32 x 17, opcodes 00 to 1F in order):

```
v2.0 raw
00000 00030 00100 00050 00070 00090 00830 00900
02000 04000 06000 08000 0A000 01210 01212 00000
00210 00211 00212 00213 00614 00615 00216 00217
00218 00219 0061A 0061B 0021C 0021D 0020E 10000
```

Program ROM (128 x 12, 68 words used):

```
v2.0 raw
080 0A1 240 841 14C 240 941 14D
240 A41 14E 2E0 16F 240 B41 150
240 BC1 151 240 C41 152 240 841
0E2 A43 153 2E0 174 240 C41 0E2
943 155 083 0A2 A81 116 2E0 177
083 D02 118 2A0 139 083 D82 11A
084 0A5 0C6 0E7 802 8A3 11B 13C
180 1A8 1C4 361 803 6A0 740 53B
11D 18F 11F F80
```

Data RAM (32 x 4):

```
v2.0 raw
7 3 5 C B A 9 3 1 2 3 4 20*0
```

Appendix C: Files Submitted

File	Description
25JE0948_CPU.circ	Logisim circuit with subcircuits ALU, REGFILE, CONTROL and CPU_TOP and memory contents loaded
rom.txt, ram.txt, control_rom.txt	Memory images
This report	Architecture, ISA, control truth table, datapath, enhancements, verification records and screenshots